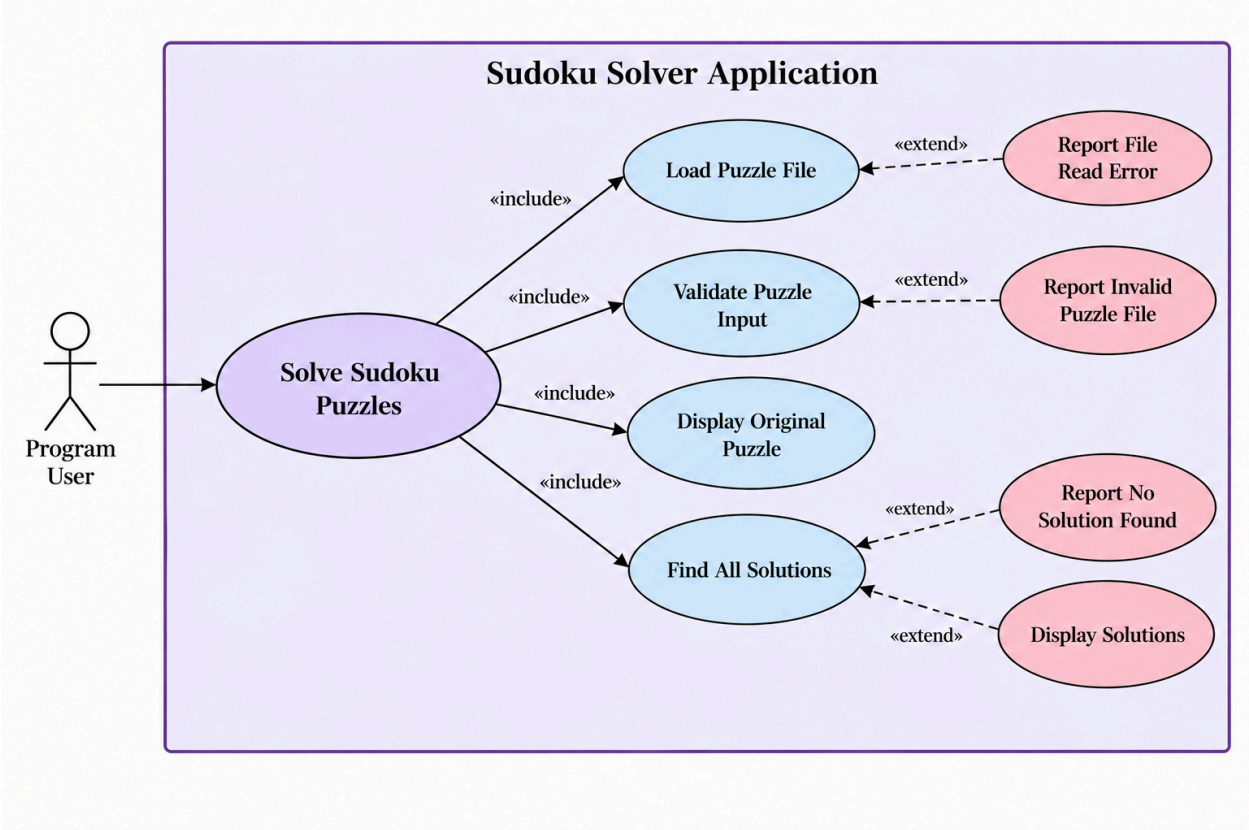


DIAGRAM CREATED BY CHATGPT

A. Solve Sudoku Puzzles
B. This use case is interacted with by the program user to read unsolved sudoku puzzles and use recursive backtracking in order to solve them. In order to do so, it will validate puzzle input, display both original puzzle and found solutions, as well as report results, including if no solutions were found.
C. 1. User executes the Sudoku Solver Application
   2. System identifies all puzzle files
   3. System reads a found puzzle files
   4. CONDITIONAL: If fails to read file, reports file read error
   5. System validates input for file
   6. CONDITIONAL: If an error is found, reports an invalid puzzle file
   7. System formats and prints the original, unsolved puzzle
   8. System finds every solution to the puzzle
   9. CONDITIONAL: If system finds solutions, formats and prints all possible solutions
   10. CONDITIONAL: If no solution is found, reports no solutions
   11. Repeats steps 3-10 for all remaining puzzle files
   12. Program terminates
D. 1. User Executes the Application

- Upon execution, the system analyzes the file names and identifies all Sudoku puzzles to be solved.

2. System Reads A Puzzle File
   - The system attempts to read the file, processing digits 1-9 as valid data for occupied cells, and an underscore as an empty cell. The system then formats the puzzle data into a 9x9 grid.
3. CONDITIONAL → System Fails to Read File and Reports File Read Error
   - If the system cannot access, open, or read the file, it will display an error message stating that there was an error reading the file. The system will then skip the failed file and continue the process with the remaining files.
4. System Validates File Input
   - System confirms that the provided puzzle is a 9x9 grid with 81 cells. The system also verifies that all provided numbers are valid and don't break the game's rules.
5. CONDITIONAL → System Fails to Validate File and Reports Invalid File Error
   - If the system finds any invalid characters, starting numbers, or grid size, it will display an error message stating that the file is invalid. The system will then skip the invalid file and continue running with the remaining files.
6. System Prints Original, Unsolved Puzzle
   - The system formats the data into a readable 9x9 grid with 81 cells. The format is the same as the file's, using numeric data 1-9 for each occupied cell and underscores for empty cells. The system will print the formatted grid to the terminal.
7. System Finds All Solutions
   - Using brute-force recursive backtracking, the system finds all possible solutions to the puzzle. It keeps going until all solutions are found, or until it determines that there are no possible solutions.
8. CONDITIONAL: System Displays Each Solution
   - If at least one solution is found, for each found solution, the system organizes the data into a readable 9x9 grid with 81 cells. The format is the same as the file's, using numeric data for all cells. The system will print the formatted grids to the terminal. Each solution is printed separately.
9. CONDITIONAL: System Reports No Solutions
   - If the system has tested all possibilities without finding a valid solution, it will display "No solution found". The system then moves on to the remaining puzzles.
10. System Processes Remaining Files
    - For all of the remaining puzzle files, the system repeats the reading, validation, printing, and solving steps. Keeps repeating until all files have been processed.
11. Program Terminates
    - Once all puzzles have been attempted to be read, processed, and solved, the program ends.